

PROYECTO: SERVICIOS TÉCNICOS

ETAPA 9 – INTEGRACIÓN FINAL DE LA APLICACIÓN

Dashboard • Navegación • Validaciones • Manejo de errores • Estilos • Flujo integral

Propósito de la etapa: integrar los módulos desarrollados para que el usuario pueda iniciar sesión y trabajar de forma continua con Clientes, Técnicos y Servicios, utilizando navegación protegida, validaciones, mensajes claros, estilos consistentes y manejo básico de errores.

1. Objetivo general

Al finalizar esta práctica, el estudiante tendrá una versión integrada del sistema “Servicios Técnicos”, en la que el flujo completo funcione desde el inicio de sesión hasta el registro y consulta de servicios.

2. Objetivos específicos

- Integrar Dashboard y navegación entre los módulos.
- Mantener las rutas protegidas mediante RutaProtegida.
- Agregar validaciones básicas a formularios.
- Mostrar mensajes de éxito, advertencia y error.
- Agregar estados de carga para evitar acciones duplicadas.
- Aplicar estilos visuales consistentes a las páginas.
- Probar el flujo integral Login → Dashboard → Clientes/Técnicos → Servicios.
- Preparar una versión estable para la etapa final de seguridad, documentación y publicación.

3. Requisitos previos

- Etapa 8 terminada.
- MySQL funcionando y base de datos tecnico disponible.
- Backend Flask ejecutándose correctamente.
- Frontend React/Vite ejecutándose correctamente.
- Módulos Clientes, Técnicos y Servicios funcionando.
- Thunder Client disponible en VS Code.
- Git configurado para el proyecto.



4. Estructura esperada del proyecto

```
Servicios-Tecnicos/
├── backend/
│   ├── app/
│   │   ├── main.py
│   │   ├── config/
│   │   │   └── conexion.py
│   │   └── routes/
│   │       ├── clientes.py
│   │       ├── tecnicos.py
│   │       └── servicios.py
│   └── venv/
├── frontend/
│   ├── src/
│   │   ├── pages/
│   │   │   ├── Login.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── Clientes.jsx
│   │   │   ├── Tecnicos.jsx
│   │   │   └── Servicios.jsx
│   │   ├── services/
│   │   │   └── api.js
│   │   ├── components/
│   │   │   └── RutaProtegida.jsx
│   │   └── App.jsx
│   └── package.json
└── README.md
```

5. Actividad 1 – Levantar todo el sistema

Abre dos terminales. En la primera inicia Flask:

```
cd backend
venv\Scripts\activate
python -m app.main
```

En la segunda inicia React:

```
cd frontend
npm run dev
```

Comprueba que puedas abrir el frontend y que el backend responda. Si el puerto cambia, utiliza el puerto mostrado por Flask/Vite.



6. Actividad 2 – Revisar el Dashboard

El Dashboard debe funcionar como punto central de navegación. Verifica que tenga enlaces a:

- Clientes
- Técnicos
- Servicios
- Cerrar sesión

Ejemplo de navegación con React Router:

```
<nav>
  <Link to="/dashboard">Inicio</Link>
  <Link to="/clientes">Clientes</Link>
  <Link to="/tecnicos">Técnicos</Link>
  <Link to="/servicios">Servicios</Link>
  <button onClick={cerrarSesion}>Cerrar sesión</button>
</nav>
```

No es necesario copiar exactamente este diseño. Lo importante es que la navegación sea clara y funcione.

7. Actividad 3 – Verificar las rutas protegidas

Todas las páginas internas deben estar protegidas. El usuario no debe entrar directamente a Clientes, Técnicos o Servicios sin iniciar sesión.

```
<Route
  path="/clientes"
  element={
    <RutaProtegida>
      <Clientes />
    </RutaProtegida>
  }
/>

<Route
  path="/tecnicos"
  element={
    <RutaProtegida>
      <Tecnicos />
    </RutaProtegida>
  }
/>

<Route
  path="/servicios"
```




```
  element={
    <RutaProtegida>
      <Servicios />
    </RutaProtegida>
  }
/>
```

Prueba: cierra sesión, escribe manualmente /clientes en el navegador y comprueba que el sistema no permita acceder al módulo.

8. Actividad 4 – Validaciones en formularios

Antes de enviar datos al backend, valida los campos principales.

- Cliente: obligatorio.
- Técnico: obligatorio.
- Descripción del servicio: obligatoria y no vacía.
- Estado: obligatorio.
- Fecha: obligatoria.
- Clientes y técnicos: nombre no vacío y datos mínimos razonables según la estructura real de la base de datos.

Ejemplo de validación en Servicios.jsx:

```
if (!clienteId || !tecnicoId || !descripcion.trim() || !estado || !fecha) {
  setMensaje("Completa todos los campos.");
  return;
}
```

La validación del frontend mejora la experiencia, pero el backend también debe validar los datos antes de guardarlos.

9. Actividad 5 – Manejo de errores y mensajes

Evita que un error de API deje la pantalla sin explicación. Usa try/catch y muestra mensajes al usuario.

```
try {
  await crearServicio(datos);
  setMensaje("Servicio registrado correctamente.");
} catch (error) {
  console.error(error);
  setMensaje("No fue posible guardar el servicio.");
}
```

Aplica el mismo criterio a Clientes, Técnicos y Servicios. Los mensajes deben ser breves y comprensibles.



10. Actividad 6 – Estados de carga

Agrega una variable de estado para indicar cuándo se está procesando una operación.

```
const [cargando, setCargando] = useState(false);

const guardar = async () => {
  setCargando(true);
  try {
    // llamada a la API
  } finally {
    setCargando(false);
  }
};
```

Ejemplo de botón:

```
<button type="submit" disabled={cargando}>
  {cargando ? "Guardando..." : "Guardar"}
</button>
```

Esto evita que el usuario haga clic varias veces mientras la API está procesando la solicitud.

11. Actividad 7 – Estilos consistentes

Crea o centraliza un archivo CSS general para que las páginas compartan una apariencia uniforme. Por ejemplo:

```
/* frontend/src/index.css */

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
}

.contenedor {
  width: min(1100px, 92%);
  margin: 0 auto;
}

.tarjeta {
  background: white;
  padding: 20px;
  margin: 20px 0;
  border-radius: 10px;
}

.boton {
```




```
  padding: 8px 14px;
  border: 0;
  border-radius: 6px;
  cursor: pointer;
}

.tabla {
  width: 100%;
  border-collapse: collapse;
}

.tabla th,
.tabla td {
  padding: 10px;
  border: 1px solid #ddd;
}
```

Puedes mejorar los estilos, pero mantén legibilidad, contraste, espacios adecuados y botones identificables.

12. Actividad 8 – Mejorar la experiencia de Clientes, Técnicos y Servicios

- Mostrar título y descripción breve de cada módulo.
- Colocar el formulario en una tarjeta o sección claramente identificada.
- Mostrar la tabla debajo del formulario.
- Distinguir los botones Nuevo/Guardar, Editar y Eliminar.
- Confirmar antes de eliminar un registro.
- Después de guardar o eliminar, actualizar la lista.
- Limpiar el formulario después de una operación exitosa.
- Mantener visible el mensaje de resultado.

Ejemplo de confirmación:

```
if (!window.confirm("¿Deseas eliminar este registro?")) {
  return;
}
```

13. Actividad 9 – Confirmar la integración de api.js

Revisa que el archivo de servicios centralice las peticiones al backend. Por ejemplo:

```
export async function obtenerServicios() {
  const respuesta = await fetch("http://127.0.0.1:5000/api/servicios");
  if (!respuesta.ok) throw new Error("Error al obtener servicios");
```




```
    return respuesta.json();
}
```

Aplica la misma idea a clientes, técnicos y operaciones POST, PUT y DELETE. No repitas innecesariamente la URL y lógica de conexión en cada componente.

14. Actividad 10 – Flujo integral de prueba

Realiza exactamente el siguiente recorrido:

1. Abrir la aplicación.
2. Iniciar sesión con un usuario válido.
3. Comprobar que aparece el Dashboard.
4. Entrar a Clientes y registrar un cliente de prueba.
5. Editar el cliente y verificar el cambio.
6. Entrar a Técnicos y registrar un técnico de prueba.
7. Editar el técnico y verificar el cambio.
8. Entrar a Servicios.
9. Comprobar que los selectores muestran clientes y técnicos.
10. Registrar un servicio relacionando el cliente y el técnico.
11. Comprobar que la tabla muestra los nombres relacionados.
12. Editar el servicio y cambiar su estado.
13. Eliminar el registro de prueba.
14. Cerrar sesión.
15. Intentar acceder directamente a una ruta protegida y comprobar que el sistema lo impide.

15. Pruebas con Thunder Client

Antes de considerar terminada la integración, confirma que el backend continúa funcionando de manera independiente.

Método	Endpoint	Resultado esperado	Comprobación
GET	/api/clientes	200	Lista de clientes
POST	/api/clientes	201/200	Cliente creado
GET	/api/tecnicos	200	Lista de técnicos
POST	/api/tecnicos	201/200	Técnico creado
GET	/api/servicios	200	Servicios con relaciones
POST	/api/servicios	201/200	Servicio creado
PUT	/api/servicios/ID	200	Servicio actualizado
DELETE	/api/servicios/ID	200/204	Servicio eliminado



16. Pruebas de errores

- Enviar un formulario vacío y comprobar que aparece una validación.
- Intentar crear un servicio sin cliente.
- Intentar crear un servicio sin técnico.
- Probar un endpoint incorrecto y comprobar que el frontend no se bloquee.
- Apagar temporalmente Flask y observar el mensaje de error del frontend.
- Volver a iniciar Flask y comprobar que la aplicación se recupera.

17. Criterios de aceptación de la etapa

Elemento	Criterio	Cumplido
Login	Permite iniciar/cerrar sesión y dirige al Dashboard.	☐
Navegación	Los enlaces del menú llevan al módulo correcto.	☐
Protección	Las rutas internas requieren autenticación.	☐
Clientes	Consulta, alta, edición y eliminación funcionan.	☐
Técnicos	Consulta, alta, edición y eliminación funcionan.	☐
Servicios	CRUD y relaciones Cliente/Técnico funcionan.	☐
Validaciones	Los formularios no envían datos vacíos.	☐
Errores	Se muestran mensajes cuando una operación falla.	☐
Carga	Los botones evitan operaciones duplicadas durante peticiones.	☐
Diseño	Las pantallas tienen una apariencia consistente.	☐
Flujo integral	Login → Dashboard → módulos → cierre de sesión funciona.	☐

18. Evidencias a entregar

- Captura del Login funcionando.
- Captura del Dashboard y su menú de navegación.
- Captura del módulo Clientes.
- Captura del módulo Técnicos.
- Captura del módulo Servicios mostrando Cliente y Técnico relacionados.
- Captura de una validación de formulario.
- Captura de un mensaje de error o de operación exitosa.
- Captura de Thunder Client con al menos una petición GET y una POST/PUT/DELETE.
- Captura de la consola o terminal mostrando el backend activo sin errores.
- Enlace o evidencia del commit realizado en Git.



19. Preguntas de análisis

1. ¿Por qué conviene tener un Dashboard como punto central de la aplicación?

Respuesta: _____

_____

2. ¿Qué diferencia existe entre validar datos en React y validarlos en Flask?

Respuesta: _____

_____

3. ¿Por qué es importante mostrar mensajes cuando falla una petición HTTP?

Respuesta: _____

_____

4. ¿Qué problema evita deshabilitar un botón mientras se procesa una petición?

Respuesta: _____

_____

5. ¿Qué ventaja tiene centralizar las peticiones HTTP en api.js?

Respuesta: _____

_____

6. ¿Qué podría ocurrir si las rutas de Clientes, Técnicos y Servicios no estuvieran protegidas?

Respuesta: _____

_____

7. ¿Cómo se comprueba que la relación Cliente–Técnico–Servicio está funcionando correctamente?

Respuesta: _____

_____



20. Git – Guardar la versión integrada

```
git status
git add .
git commit -m "Integra dashboard, validaciones y flujo completo"
git push
```

Si trabajas con una rama específica, realiza el push a la rama correspondiente. Comprueba en GitHub que el commit aparezca.

21. Checklist del estudiante

- ✓ Puedo iniciar sesión.
- ✓ El Dashboard aparece después del login.
- ✓ Puedo navegar a Clientes.
- ✓ Puedo navegar a Técnicos.
- ✓ Puedo navegar a Servicios.
- ✓ Las rutas internas están protegidas.
- ✓ Los formularios validan campos obligatorios.
- ✓ Se muestran mensajes de éxito/error.
- ✓ Los botones muestran estado de carga cuando corresponde.
- ✓ Los estilos son consistentes.
- ✓ El flujo CRUD de Clientes funciona.
- ✓ El flujo CRUD de Técnicos funciona.
- ✓ El flujo CRUD de Servicios funciona.
- ✓ Las relaciones Cliente/Técnico se muestran correctamente.
- ✓ Puedo cerrar sesión.
- ✓ Las pruebas con Thunder Client son satisfactorias.
- ✓ Realicé el commit y push de la etapa.

22. Solución de problemas frecuentes

Problema	Revisión recomendada
El Dashboard no aparece	Revisa la ruta /dashboard, el componente y la navegación después del login.
Una ruta protegida permite entrar sin login	Revisa RutaProtegida y la condición utilizada para comprobar la sesión.
Los datos no cargan	Comprueba que Flask esté activo, la URL de api.js y la consola del navegador.
Error CORS	Revisa la configuración CORS del backend y que frontend y backend estén usando los puertos esperados.
No se muestran nombres de Cliente/Técnico	Revisa los JOIN del endpoint de servicios y los nombres de propiedades recibidas por React.
Se duplican registros	Revisa el estado de carga y evita múltiples envíos del formulario.



El CSS no se aplica	Comprueba la importación del archivo CSS y los nombres de las clases.
DELETE no funciona	Prueba primero el endpoint en Thunder Client y confirma que el ID exista.